



PERÚ

Ministerio
de Educación

INSTITUTO DE EDUCACIÓN SUPERIOR TECNOLÓGICO PÚBLICO

“DANIEL ALCIDES CARRIÓN”

COMPUTACIÓN E INFORMÁTICA

MODULO II

DESARROLLO DE SOFTWARE Y GESTIÓN DE BASE DE DATOS



LÓGICA DE PROGRAMACIÓN

Gilber BASILIO ROBLES

2017

LÓGICA DE PROGRAMACIÓN

Es la habilidad de pensar de manera razonada, sistemática y ordenada que nos hace capaces de inferir algoritmos abstractos y soluciones a problemas de negocios, susceptibles de programación.

Un joven que aspiraba a entrar a la marina acude a una entrevista y la primera pregunta que le hacen es: ¿Sabe nadar? A lo que el sorprendido responde: ¿Y para qué tienen los barcos?

¿Cómo podría aplicarse este chiste a la informática?



En general un problema de información es posible entenderlo, analizarlo y descomponerlo en todos sus componentes o partes que de una u otra manera intervienen tanto en su planteamiento como en su solución.

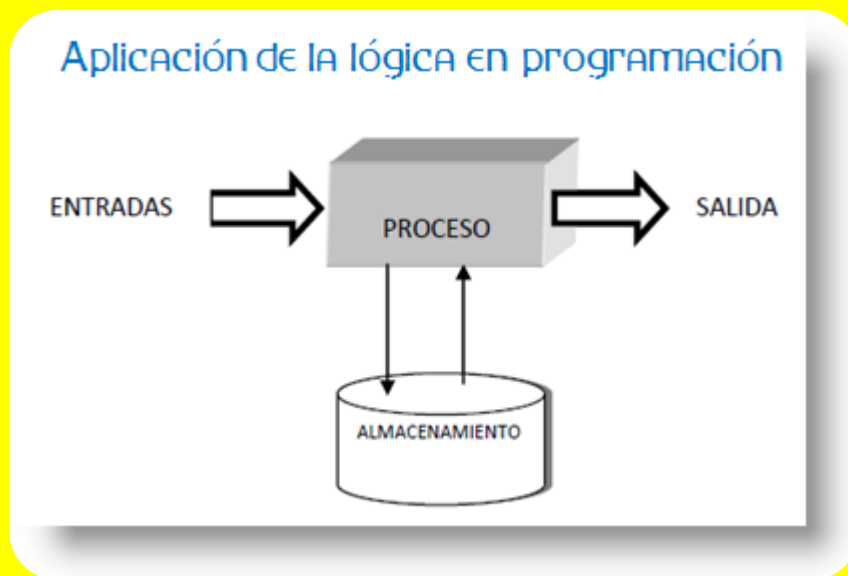
Una herramienta rápida que nos permite descomponer en partes un problema para su solución, es el llamado modelo de solución, este consiste de una pequeña caja que contiene los tres elementos más básicos en que se puede descomponer cualquier problema sencillo de información, estas tres partes son:

- ✚ **LA PRIMERA PARTE:** son todos los datos que el computador ocupa para resolver el problema, estos datos son almacenados internamente en la memoria del computador en las llamadas variables de entrada.

- ✚ **LA SEGUNDA PARTE:** son todas las operaciones generalmente algebraicas necesarias para solucionar el problema, generalmente esta parte del modelo es una *formula* (o igualdad matemática, Ejemplo: $X = y + 5$).
- ✚ **LA TERCERA PARTE:** es el resultado o solución del problema que generalmente se obtiene de la parte de operaciones del modelo y dichos datos están almacenados en las llamadas variables de salida.

En resumen, para todo problema sencillo de información es necesario plantearse las siguientes preguntas:

- ¿Qué datos ocupa conocer el computador para resolver el problema y en cuales variables de entrada se van a almacenar?
- ¿Qué procesos u operaciones debe realizar el computador para resolver el problema planteado?
- ¿Qué información o variables de salida se van a desplegar en pantalla para responder al problema planteado originalmente?



Ejemplo: CALCULAR EL ÁREA DE UN CÍRCULO CON LA FORMULA: $AREA = PI * RADIO^2$

INFORMACIÓN Y DATOS.

INFORMACIÓN. Información, significado que el hombre asigna a los datos por medio de las convenciones utilizadas en su representación.

INFORMÁTICA. Ciencia que estudia el tratamiento racional de la *información* por medio de *máquinas automáticas*. La palabra Informática está formada por la contracción de los vocablos **INFORM**ación y automática

DATOS. Los **datos** son conjuntos de símbolos utilizados para expresar o representar un valor numérico, un hecho, un objeto o una idea, en la forma adecuada para ser objeto de tratamiento.

Los datos son los objetos sobre los que opera la computadora según las órdenes de un programa, cuya acción se refleja en cambios en los mismos.



Los datos pueden ser captados directamente por la computadora (detectando electrónicamente un sonido, una temperatura, etc.) o pueden ser dados, representándolos en forma de letras y números (grafismos). En Informática los grafismos usados para representar los datos están constituidos por:

- ⊙ Caracteres numéricos (las diez cifras decimales),
- ⊙ Caracteres alfabéticos
- ⊙ Caracteres especiales (símbolos ortográficos, aritméticos y otros).

LA LÓGICA COMO ASPECTO FUNDAMENTAL DE LA PROGRAMACIÓN.

Es la capacidad de pensar racionalmente acerca de soluciones alternativas y los resultados de aplicarlas y, por lo tanto, de hacer elecciones inteligentes.

DEFINICIONES DE LÓGICA.

- ☞ Es el estudio crítico del razonamiento y tiene un valor teórico y práctico.
- ☞ Es el estudio de los métodos y principios usados al distinguir entre los argumentos correctos (buenos) y los argumentos incorrectos (malos).
- ☞ En un sentido amplio, es el estudio del correcto razonamiento. La lógica se remonta a la época de Aristóteles en el siglo IV antes de Cristo. En lógica, Aristóteles desarrolló reglas para establecer un razonamiento encadenado

que, si se respetaban, no producirían nunca falsas conclusiones si la reflexión partía de premisas verdaderas (reglas de validez). En el ejemplo más famoso, "Todos los humanos son mortales" y "Todos los griegos son humanos", se llega a la conclusión válida de que "Todos los griegos son mortales"

¿QUÉ ES LÓGICA?

- 🖱️ Lógica es la ciencia que estudia el proceso de razonar.
- 🖱️ Razonar es ordenar las ideas para llegar a una conclusión.
- 🖱️ Procesar es ejecutar una serie de actividades interrelacionados.

LÓGICA DEDUCTIVA. Es la que se encarga de determinar la validez o invalidez de los argumentos. Permite proporcionar la simbología que nos servirá para facilitar el desarrollo de la capacidad de análisis de problemas, para obtener una solución posible.

Argumentos. Cuando el razonamiento se expresa con palabras, recibe el nombre de "argumento". Un argumento es un grupo cualquiera de proposiciones o enunciados que forman premisas y conclusiones. Este puede constar de varias premisas, pero de una sola conclusión.

PREMISAS Y CONCLUSIÓN.

Las premisas de un argumento son proposiciones afirmadas como fundamento o razones para aceptar una conclusión. La conclusión es la proposición afirmada que se basa en las otras proposiciones o premisas. Una proposición puede ser premisa en un argumento y conclusión en otro.

Por ejemplo: "Todos los hombres son mortales" Aparece como premisa en el siguiente argumento:

"Todos los hombres son mortales"

"Sócrates es un hombre"

"Por lo tanto, Sócrates es mortal".

Y como conclusión en el siguiente argumento:

"Todos los animales son mortales"

"Todos los hombres son animales"

"Luego, todos los hombres son mortales"

Expresiones como "por tanto", "por lo cual", "de ello se deduce", sirven para introducir la conclusión de un argumento, en tanto que "pues" y "porque" se emplean para introducir las premisas.

Otro Ejemplo:

"Todos se aburren en la conferencia".

"Ninguno de los que se aburren presta atención".

"Por consiguiente, ninguno de los asistentes está prestando atención".

Hay dos condiciones que debe satisfacer un argumento para establecer la verdad de su conclusión: Debe ser válido y todas sus premisas deben ser verdaderas.

Enunciado Simple:

Es el que no contiene otro enunciado como parte componente. Ej. "**Las rosas son rojas**"

Enunciado compuesto:

Es el que se compone de varios enunciados. Ej. "**Las rosas son rojas y las violetas son azules**". Cuando los enunciados se unen por la conjunción Y.

Enunciados conjuntos.

Cuando los enunciados se unen por el conector o, se denominan **Enunciados disyuntos**. De aquí surgen las siguientes tablas de verdad:

X	Y	And
V	V	V
V	F	F
F	V	F
F	F	F

X	Y	OR
V	V	V
V	F	V
F	V	V
F	F	F

X	NOT (X)
V	F
F	V

Tablas de Verdad

PRINCIPIOS EN LA ELABORACIÓN DE ENUNCIADOS.

Los enunciados son los planteamientos que hacemos para definir una actividad a realizar en procura de la solución del problema.

PRINCIPIOS DE LOS ENUNCIADOS:**+ ENTRADA**

- Economía de los datos
- Autosuficiencia de los datos
- Compatibilidad de los datos
- Conocimiento preestablecido

+ PROCESO

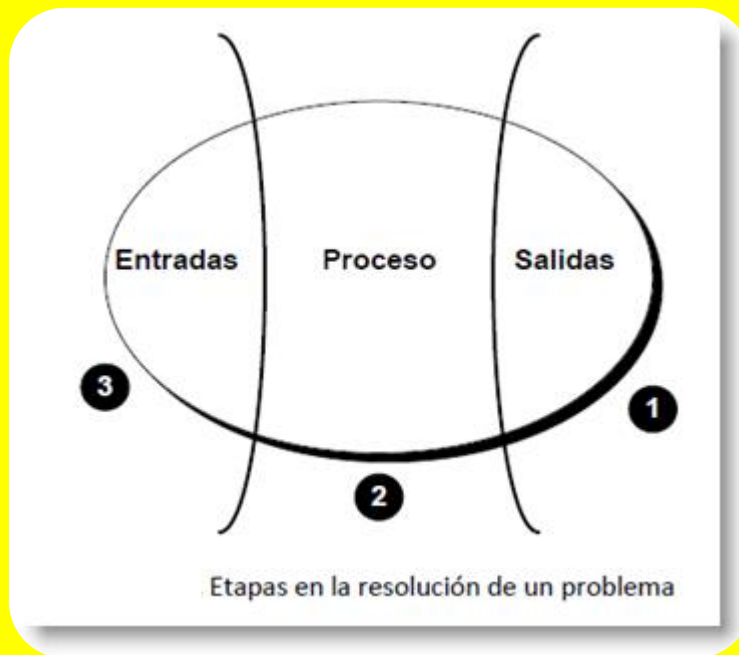
- Consistencia de resultado
- Eficiencia del proceso
- Eficacia del proceso
- Simplicidad del proceso
- Independencia de los procesos
- Reutilización de los procesos
- Rastreabilidad de las operaciones

+ SALIDA

- Cantidad y forma
- Exactitud
- Dominio de salida

COMPRENSIÓN Y ESPECIFICACIÓN DEL PROBLEMA.

- + Complejidad del problema.
- + Análisis del problema.
- + Variantes
- + Casos de prueba.

IDENTIFICACIÓN ENTRADAS, PROCESO Y SALIDAS.**Ejemplo:**

Elabore los enunciados requeridos para el cálculo de la edad de una persona.

+ ENUNCIADOS DE LAS SALIDAS:

- La edad debe expresarse en años, meses y días.
- La edad en años no debe ser menor que cero.
- La edad en meses no puede ser menor que cero.
- La edad en meses no puede ser mayor que once.
- La edad en días no puede ser menor que cero.
- La edad en días no puede ser mayor que treinta.

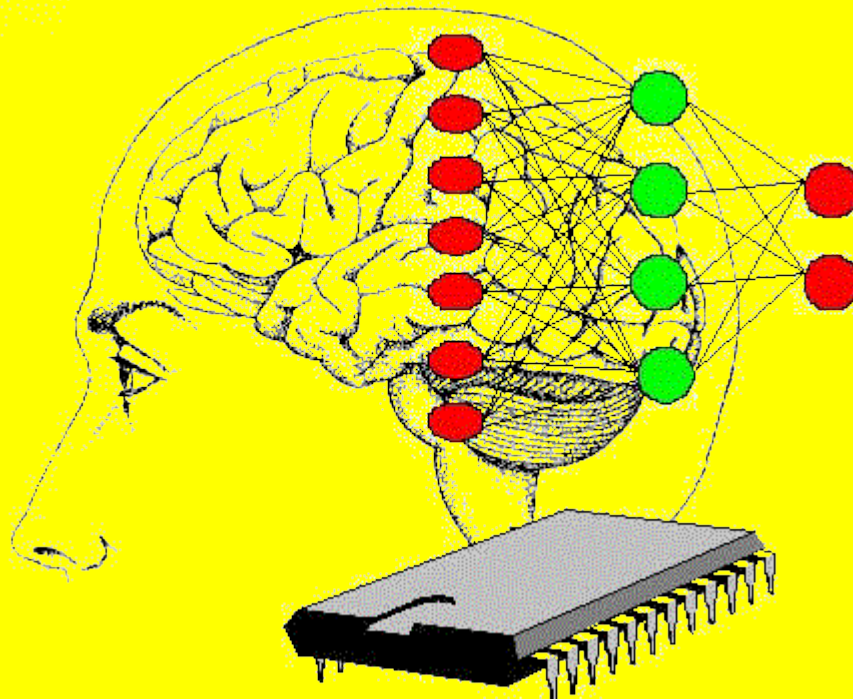
+ ENUNCIADOS DEL PROCESO

- Para calcular la edad se debe restar la fecha de nacimiento de la fecha actual.
- La fecha debe descomponerse en años, meses y días.
- Se debe comenzar restando los días.

- Si el día de nacimiento es mayor al día actual debe solicitar un mes expresado en días a los meses de la fecha actual, con lo cual se suman 30 días al día actual y se resta un mes al mes actual.
- En segunda instancia debe restarse los meses.
- Si el mes de nacimiento es mayor que el mes de la fecha actual, debe solicitar un año expresado en meses a la fecha actual, con lo cual se suman 12 meses al mes actual y se resta un año a año actual.
- Finalmente se resta los años.

ENUNCIADOS DE LA ENTRADA:

- Para el cálculo de la edad es necesario ingresar la fecha de nacimiento y la fecha actual.
- La fecha actual y de nacimiento deben ingresarse en tres números cada una que representan año, mes y día.
- La fecha de nacimiento nunca puede ser mayor que la fecha actual.
- Un año tiene 12 meses.
- Un mes tiene 28, 30 o 31 días.
- Los meses del año pueden tener valores del 1 al 12.
- Los días del mes pueden tener valores del 1 al 31.



SOLUCIÓN DE PROBLEMAS CON ALGORITMOS

¿QUÉ ES UN PROBLEMA?

Un problema se puede definir como una situación en la cual se trata de alcanzar una meta y para lograrlo se deben hallar y utilizar unos medios y unas estrategias.



LOS PROBLEMAS:

- ⊙ La mayoría de problemas tienen algunos elementos en común:
 - ✓ Un **estado inicial**;
 - ✓ Una **meta**, lo que se pretende lograr;
 - ✓ Un conjunto de **recursos**, lo que está permitido hacer y/o utilizar;
 - ✓ Un **dominio**, es decir, los conocimientos, habilidades de quien va a resolverlo.

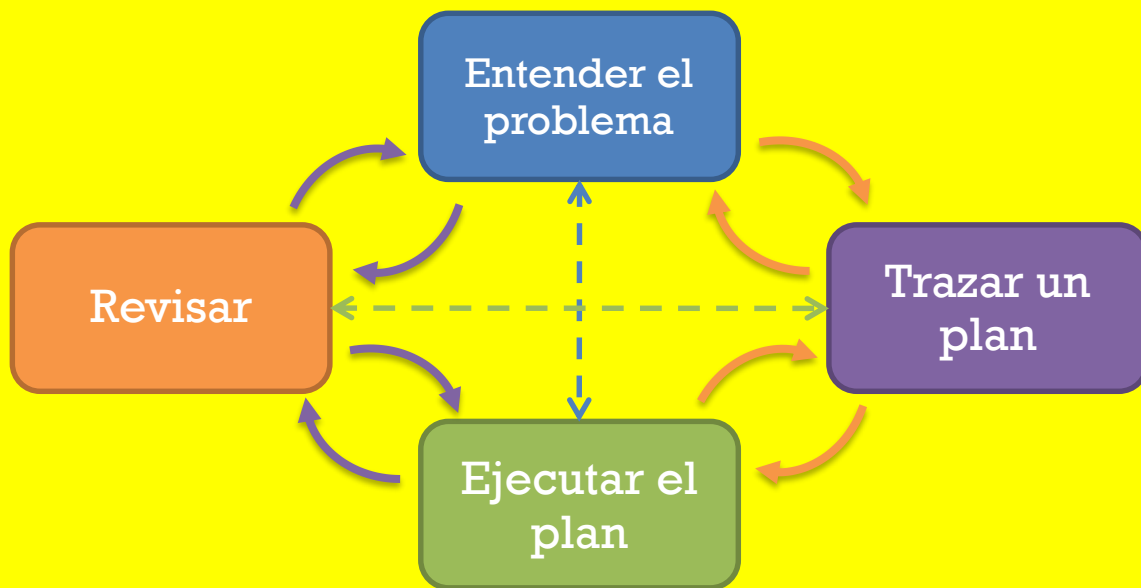
ESTRATEGIAS PARA RESOLVER PROBLEMAS

- ⊙ **Ensayo y error**: Consiste en actuar hasta que algo funcione. Puede tomar mucho tiempo y no es seguro que se llegue a una solución.
- ⊙ **Heurística**: Se basa en la utilización de reglas producto de la experiencia, para llegar a una solución.
- ⊙ **Algoritmos**: Consiste en aplicar adecuadamente una serie de pasos detallados que aseguran una solución correcta.
- ⊙ **Lluvia de ideas**: Consiste en formular soluciones viables a un problema.

PASOS PARA RESOLVER UN PROBLEMA

⊙ Un método heurístico formulado por el matemático George Polya indica que cuando se resuelven problemas, intervienen cuatro operaciones mentales:

- ✓ Entender el problema
- ✓ Trazar un plan
- ✓ Ejecutar el plan (resolver)
- ✓ Revisar

**PASO 1: ENTENDER EL PROBLEMA**

- ⊙ Leer el problema varias veces
- ⊙ ¿Qué datos me dan en el enunciado del problema?
- ⊙ ¿Cuál es la pregunta que me da el problema?
- ⊙ ¿Qué debo lograr?
- ⊙ ¿Cuál es la incógnita del problema?
- ⊙ Organizar la información
- ⊙ Agrupar los datos en categorías
- ⊙ Trazar una figura o diagrama.

PASO 2: TRAZAR EL PLAN

- ⊙ Escoger y decidir las operaciones a efectuar.
- ⊙ Eliminar los datos inútiles.
- ⊙ Descomponer el problema en otros más pequeños.

PASO 3: EJECUTAR EL PLAN

- ⊙ Ejecutar en detalle cada operación.
- ⊙ Simplificar antes de calcular.
- ⊙ Realizar un dibujo o diagrama

PASO 4: ANALIZAR LA SOLUCIÓN (REVISAR)

- ⊙ Dar una respuesta completa
- ⊙ Hallar el mismo resultado de otra manera.
- ⊙ Verificar por apreciación que la respuesta es adecuada.

Ejemplo:

En un juego, el ganador obtiene una ficha roja; el segundo, una ficha azul; y el tercero, una amarilla.

Al final de varias rondas, el puntaje se calcula de la siguiente manera: Al triple de la cantidad de fichas rojas se adicionan las fichas azules y se descuenta el doble de las fichas amarillas.

Si Andrés llegó 3 veces en primer lugar, 4 veces de último y 6 veces de intermedio, ¿Qué puntaje obtuvo?

COMPRENDE (paso 1)

- ⊙ Leer detenidamente el problema
- ⊙ ¿Cuántos colores de fichas se reparten?
- ⊙ ¿Cuántas fichas rojas, azules y amarillas obtuvo Andrés?
- ⊙ ¿Qué pregunta el problema?



PLANEA (paso 2)

- ⊙ Para hallar el puntaje que obtiene Andrés por sus llegadas de primero, calcular el triple de la cantidad de fichas rojas.
- ⊙ Para hallar el puntaje por sus llegadas en segundo lugar, contar la cantidad de fichas azules.
- ⊙ Para hallar el puntaje que pierde por sus llegadas en último lugar, calcular el doble de la cantidad de fichas amarillas.
- ⊙ Para hallar el puntaje total, calcular la suma de los puntajes por las fichas rojas y azules, restarle los puntos de las fichas amarillas.

**RESUELVE (paso 3)**

- ⊙ Por tres fichas rojas: $3 \times 3 = 9$ puntos
- ⊙ Por seis fichas azules: $6 = 6$ puntos
- ⊙ Por cuatro fichas amarillas: $4 \times 2 = 8$ puntos
- ⊙ Para obtener el puntaje final de Andrés, sumar los puntos obtenidos con las fichas rojas y azules ($9 + 6 = 15$ puntos) y de este resultado restar los puntos representados por las fichas amarillas ($15 - 8 = 7$ puntos).

**REVISA**

- ⊙ El puntaje que obtuvo Andrés es 7 puntos.
- ⊙ Verificar las operaciones y comparar los cálculos con la solución estimada.



UNIDADES DE MEDIDA DE INFORMACIÓN

Las unidades de medida se definen por convenciones internacionales. Por deducción lógica la unidad fundamental de la masa debería ser el gramo, pero es una cantidad muy pequeña, así que se escogió el kilogramo debido a que es más común y práctico utilizarlo. En el caso de la información, sucede algo similar, la unidad de medida de la información es el bit, pero por cuestiones de utilidad se utiliza el "Byte" que significa octeto. Puede abreviarse como b ó B, pero aún no se ha estandarizado su forma de representarlo, por lo que en este sitio utilizamos la B para referirnos al Byte, siendo correcto también abreviarlo con b (byte).

La computadora trabaja con en el sistema binario, que se basa solo en 2 dígitos: El cero (0) y el uno (1). Un bit es simplemente un cero ó un uno, pero la computadora trabaja con conjuntos de ocho combinaciones de ceros y unos, a esto se le denomina Byte (octeto).

Bit

Un Bit es el acrónimo de Binary digit (dígito binario). **Es la unidad mínima de información** empleada en informática, en cualquier dispositivo digital, o en la teoría de la información. Con él, podemos representar dos valores cuales quiera, como verdadero o falso, abierto o cerrado, blanco o negro, norte o sur, masculino o femenino, rojo o azul, etc.

Múltiplos del Bit:

Medida	Simbología	Equivalencia	Equivalente en Bytes
byte	b	8 bits	1 byte
kilobyte	Kb	1024 bytes	1 024 bytes
megabyte	MB	1024 KB	1 048 576 bytes
gigabyte	GB	1024 MB	1 073 741 824 bytes
terabyte	TB	1024 GB	1 099 511 627 776 bytes
Petabyte	PB	1024 TB	1 125 899 906 842 624 bytes
Exabyte	EB	1024 PB	1 152 921 504 606 846 976 bytes
Zetabyte	ZB	1024 EB	1 180 591 620 717 411 303 424 bytes
Yottabyte	YB	1024 ZB	1 208 925 819 614 629 174 706 176 bytes
Brontobyte	BB	1024 YB	1 237 940 039 285 380 274 899 124 224 bytes
Geopbyte	GB	1024 BB	1 267 650 600 228 229 401 496 703 205 376 bytes

SISTEMAS DE NUMERACIÓN

Un sistema de numeración es un conjunto de símbolos y reglas que permiten representar datos numéricos. Los sistemas de numeración actuales son sistemas posicionales, que se caracterizan porque **un símbolo tiene distinto valor según la posición que ocupa en la cifra.**

Binario	0 1
Octal	0 1 2 3 4 5 6 7
Decimal	0 1 2 3 4 5 6 7 8 9
Hexadecimal	0 1 2 3 4 5 6 7 8 9 A B C D E F

SISTEMA DE NUMERACIÓN DECIMAL:

El sistema de numeración que utilizamos habitualmente es el decimal, que se compone de diez símbolos o dígitos (0, 1, 2, 3, 4, 5, 6, 7, 8 y 9) a los que otorga un valor dependiendo de la posición que ocupen en la cifra: unidades, decenas, centenas, millares, etc.

El valor de cada dígito está asociado al de una potencia de base 10, número que coincide con la cantidad de símbolos o dígitos del sistema decimal, y un exponente igual a la posición que ocupa el dígito menos uno, contando desde la derecha.

En el sistema decimal el número **528**, por ejemplo, significa:

5 centenas + 2 decenas + 8 unidades, es decir:

$5 \cdot 10^2 + 2 \cdot 10^1 + 8 \cdot 10^0$ o, lo que es lo mismo:

$$500 + 20 + 8 = 528$$

En el caso de números con decimales, la situación es análoga, aunque, en este caso, algunos exponentes de las potencias serán negativos, concretamente el de los dígitos colocados a la derecha del separador decimal. Por ejemplo, el número **8245,97** se calcularía como:

8 millares + 2 centenas + 4 decenas + 5 unidades + 9 décimos + 7 céntimos

$$8 \cdot 10^3 + 2 \cdot 10^2 + 4 \cdot 10^1 + 5 \cdot 10^0 + 9 \cdot 10^{-1} + 7 \cdot 10^{-2}, \text{ es decir:}$$

$$8000 + 200 + 40 + 5 + 0,9 + 0,07 = 8245,97$$

SISTEMA DE NUMERACIÓN BINARIO.

El sistema de numeración binario utiliza sólo dos dígitos, el cero (0) y el uno (1).

En una cifra binaria, cada dígito tiene distinto valor dependiendo de la posición que ocupe. El valor de cada posición es el de una potencia de **base 2**, elevada a un exponente igual a la posición del dígito menos uno.

De acuerdo con estas reglas, el número binario 1011 tiene un valor que se calcula así:

$$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0, \text{ es decir:}$$

$$8 + 0 + 2 + 1 = 11$$

y para expresar que ambas cifras describen la misma cantidad lo escribimos así:

$$1011_2 = 11_{10}$$

CONVERSIÓN ENTRE NÚMEROS DECIMALES Y BINARIOS:

Convertir un número decimal al sistema binario es muy sencillo: basta con realizar divisiones sucesivas por 2 y escribir los restos obtenidos en cada división en orden inverso al que han sido obtenidos.

Por ejemplo, para convertir al sistema binario el número 77_{10} haremos una serie de divisiones que arrojarán los restos siguientes:

$$77 : 2 = 38 \text{ Resto: } 1$$

$$38 : 2 = 19 \text{ Resto: } 0$$

$$19 : 2 = 9 \text{ Resto: } 1$$

$$9 : 2 = 4 \text{ Resto: } 1$$

$$4 : 2 = 2 \text{ Resto: } 0$$

$$2 : 2 = 1 \text{ Resto: } 0$$

$$1 : 2 = 0 \text{ Resto: } 1$$

y, tomando los restos en orden inverso obtenemos la cifra binaria:

$$77_{10} = 1001101_2$$

Ejercicio:

Expresa, en código binario, los números decimales siguientes: **191, 25, 67, 99, 135, 276.**

CONVERSIÓN DE BINARIO A DECIMAL.

El proceso para convertir un número del sistema binario al decimal es aún más sencillo; basta con desarrollar el número, teniendo en cuenta el valor de cada dígito en su posición, que es el de una potencia de 2, cuyo exponente es 0 en el bit situado más a la derecha, y se incrementa en una unidad según vamos avanzando posiciones hacia la izquierda.

Por ejemplo, para convertir el número binario **1010011₂** a decimal, lo desarrollamos teniendo en cuenta el valor de cada bit:

$$1*2^6 + 0*2^5 + 1*2^4 + 0*2^3 + 0*2^2 + 1*2^1 + 1*2^0 = 83$$

$$1010011_2 = 83_{10}$$

Ejercicio:

Expresa, en el sistema decimal, los siguientes números binarios: **110111, 111000, 010101, 101010, 1111110**

SISTEMA DE NUMERACIÓN OCTAL

El inconveniente de la codificación binaria es que la representación de algunos números resulta muy larga. Por este motivo se utilizan otros sistemas de numeración que resulten más cómodos de escribir: el sistema octal y el sistema hexadecimal.

En el sistema de numeración octal, los números se representan mediante **ocho** dígitos diferentes: 0, 1, 2, 3, 4, 5, 6 y 7. Cada dígito tiene, naturalmente, un valor distinto dependiendo del lugar que ocupen. El valor de cada una de las posiciones viene determinado por las potencias de base 8.

Por ejemplo, el número octal **273₈** tiene un valor que se calcula así:

$$2*8^3 + 7*8^2 + 3*8^1 = 2*512 + 7*64 + 3*8 = 1496_{10}$$

$$273_8 = 1496_{10}$$

CONVERSIÓN DE UN NÚMERO DECIMAL A OCTAL.

La conversión de un número decimal a octal se hace con la misma técnica que ya hemos utilizado en la conversión a binario, mediante divisiones sucesivas **por 8** y colocando los restos obtenidos en **orden inverso**. Por ejemplo, para escribir en octal el número decimal 122_{10} tendremos que hacer las siguientes divisiones:

$$122 / 8 = 15 \quad \text{Resto: } 2$$

$$15 / 8 = 1 \quad \text{Resto: } 7$$

$$1 / 8 = 0 \quad \text{Resto: } 1$$

Tomando los restos obtenidos en orden inverso tendremos la cifra octal:

$$122_{10} = 172_8$$

Ejercicio:

Convierte los siguientes números decimales en octales: 63_{10} , 513_{10} , 119_{10}

CONVERSIÓN OCTAL A DECIMAL.

La conversión de un número octal a decimal es igualmente sencilla, conociendo el peso de cada posición en una cifra octal. Por ejemplo, para convertir el número 237_8 a decimal basta con desarrollar el valor de cada dígito:

$$2 \cdot 8^2 + 3 \cdot 8^1 + 7 \cdot 8^0 = 128 + 24 + 7 = 159_{10}$$

$$237_8 = 159_{10}$$

Ejercicio:

Convierte al sistema decimal los siguientes números octales: 45_8 , 125_8 , 625_8

SISTEMA DE NUMERACIÓN HEXADECIMAL.

En el sistema hexadecimal los números se representan con dieciséis símbolos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E y F. Se utilizan los caracteres A, B, C, D, E y F representando las cantidades decimales 10, 11, 12, 13, 14 y 15 respectivamente, porque no hay dígitos mayores que 9 en el sistema decimal. El valor de cada uno de

estos símbolos depende, como es lógico, de su posición, que se calcula mediante potencias de base 16.

Calculemos, a modo de ejemplo, el valor del número hexadecimal **1A3F**₁₆:

$$1A3F_{16} = 1 \cdot 16^3 + A \cdot 16^2 + 3 \cdot 16^1 + F \cdot 16^0$$

$$1 \cdot 4096 + 10 \cdot 256 + 3 \cdot 16 + 15 \cdot 1 = 6719$$

$$1A3F_{16} = 6719_{10}$$

Ejercicio:

Expresa en el sistema decimal las siguientes cifras hexadecimales: **2BC5**₁₆, **100**₁₆, **1FF**₁₆

Ensayemos, utilizando la técnica habitual de divisiones sucesivas, la conversión de un número decimal a hexadecimal. Por ejemplo, para convertir a hexadecimal del número **1735**₁₀ será necesario hacer las siguientes divisiones:

$$1735 / 16 = 108 \quad \text{Resto: } 7$$

$$108 / 16 = 6 \quad \text{Resto: } C \text{ es decir, } 12_{10}$$

$$6 / 16 = 0 \quad \text{Resto: } 6$$

De ahí que, tomando los restos en orden inverso, resolvemos el número en hexadecimal:

$$1735_{10} = 6C7_{16}$$

Ejercicio:

Convertir al sistema hexadecimal los siguientes números decimales: **3519**₁₀, **1024**₁₀, **4095**₁₀

CONVERSIÓN DE NÚMEROS BINARIOS A OCTALES Y VICEVERSA

Observa la tabla siguiente, con los siete primeros números expresados en los sistemas decimal, binario y octal:

DECIMAL	BINARIO	OCTAL
0	000	0
1	001	1
2	010	2
3	011	3
4	100	4
5	101	5
6	110	6
7	111	7

Cada dígito de un número octal se representa con tres dígitos en el sistema binario. Por tanto, el modo de convertir un número entre estos sistemas de numeración equivale a "**expandir**" cada dígito octal a tres dígitos binarios, o en "**contraer**" grupos de tres caracteres binarios a su correspondiente dígito octal.

Por ejemplo, para convertir el número binario 101001011_2 a octal tomaremos grupos de tres bits y los sustituiremos por su equivalente octal:

$$101_2 = 5_8$$

$$001_2 = 1_8$$

$$011_2 = 3_8$$

y, de ese modo: $101001011_2 = 513_8$

Ejercicio:

Convierte los siguientes números binarios en octales: 1101101_2 , 101110_2 , 11011011_2 , 101101011_2

La conversión de números octales a binarios se hace, siguiendo el mismo método, reemplazando cada dígito octal por los tres bits equivalentes. Por ejemplo, para convertir el número octal 750_8 a binario, tomaremos el equivalente binario de cada uno de sus dígitos:

$$7_8 = 111_2$$

$$5_8 = 101_2$$

$$0_8 = 000_2$$

y, por tanto: $750_8 = 111101000_2$

Ejercicio:

Convierte los siguientes números octales en binarios: 25_8 , 372_8 , 2753_8

CONVERSIÓN DE NÚMEROS BINARIOS A HEXADECIMALES Y VICEVERSA.

Del mismo modo que hallamos la correspondencia entre números octales y binarios, podemos establecer una equivalencia directa entre cada dígito hexadecimal y cuatro dígitos binarios, como se ve en la siguiente tabla:

DECIMAL	BINARIO	HEXADECIMAL
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

La conversión entre números hexadecimales y binarios se realiza "expandiendo" o "contrayendo" cada dígito hexadecimal a cuatro dígitos binarios. Por ejemplo, para expresar en hexadecimal el número binario 101001110011_2 bastará con tomar grupos de cuatro bits, empezando por la derecha, y reemplazarlos por su equivalente hexadecimal:

$$1010_2 = A_{16}$$

$$0111_2 = 7_{16}$$

$$0011_2 = 3_{16}$$

$$\text{y, por tanto: } 101001110011_2 = A73_{16}$$

En caso de que los dígitos binarios no formen grupos completos de cuatro dígitos, se deben añadir ceros a la izquierda hasta completar el último grupo. Por ejemplo:

$$101110_2 = 00101110_2 = 2E_{16}$$

Ejercicio:

Convierte a hexadecimales los siguientes números binarios:

$$1010100101011101010_2, 111000011110000_2, 1010000111010111_2$$

La conversión de números hexadecimales a binarios se hace del mismo modo, reemplazando cada dígito hexadecimal por los cuatro bits equivalentes de la tabla. Para convertir a binario, por ejemplo, el número hexadecimal **1F6**₁₆ hallaremos en la tabla las siguientes equivalencias:

$$1_{16} = 0001_2$$

$$F_{16} = 1111_2$$

$$6_{16} = 0110_2$$

$$\text{y, por tanto: } 1F6_{16} = 000111110110_2$$

Ejercicio:

Convierte a binario los números hexadecimales siguientes: **7A5D**₁₆, **1010**₁₆, **8F8F**₁₆

HERRAMIENTAS DE PROGRAMACIÓN

ALGORITMO.

Es una secuencia lógica de pasos para la solución de un problema.

Un Algoritmo, se puede definir como una secuencia de instrucciones que representan un modelo de solución para determinado tipo de problemas. O bien como un conjunto de instrucciones que, realizadas en orden conducen a obtener la solución de un problema. Por lo tanto, ***podemos decir que es un conjunto ordenado y finito de pasos que nos permite solucionar un problema.***

Los algoritmos son independientes de los lenguajes de programación. En cada problema el algoritmo puede escribirse y luego ejecutarse en un lenguaje de programación diferente.

Criterios o características que debe satisfacer un algoritmo:

1. **Entrada.** Son cero o más cantidades las cuales son externamente sustituidas.
2. **Salida.** Al menos una cantidad es producida.
3. **Exactitud/precisión.** Cada instrucción debe ser clara y sin ambigüedad.
4. **Finito.** Terminará después de un número finito de pasos.
5. **Eficiente.** Cada instrucción puede ser verificada por una persona con una prueba manual que satisfaga los requerimientos planteados por el problema.



HERRAMIENTAS DE DESARROLLO DE ALGORITMOS.

1. Descripción Narrada
2. **Pseudocódigo**
3. **Diagramas de Flujo**
4. **Codificación**

1. DESCRIPCIÓN NARRADA.

Este algoritmo es caracterizado porque sigue un proceso de ejecución común y lógico, describiendo textualmente paso a paso cada una de las actividades a realizar dentro de una actividad determinada.

Ejemplo: Algoritmo para asistir a clases:

1. Levantarse
2. Bañarse
3. Vestirse
4. Desayunar
5. Cepillarse los dientes
6. Salir de casa
7. Tomar el autobús/Caminar
8. Llegar al IESTPDAC
9. Buscar el aula
10. Ubicarse en un asiento

2. PSEUDOCÓDIGO.

Pseudo = falso. El pseudocódigo no es realmente un código sino una imitación y una versión abreviada de instrucciones reales para las computadoras. Es una técnica para diseño de programas que permite definir las estructuras de datos, las operaciones que se aplicarán a los datos y la lógica que tendrá el programa de computadora para solucionar un determinado problema. Utiliza un pseudolenguaje muy parecido a nuestro idioma, pero que respeta las directrices y los elementos de los lenguajes de programación.

Ejemplo:

Diseñar un algoritmo que lea cuatro variables y calcule e imprima su producto, suma y media aritmética.

Inicio

Leer (a, b, c, d)

Producto = (a * b * c * d)

Suma = (a + b + c + d)

Media = (a + b + c + d) / 4

Escribir (producto, suma, media)

Fin

3. DIAGRAMAS DE FLUJO.

Son la representación gráfica de la solución algorítmica de un problema. Para diseñarlos se utilizan determinados símbolos o figuras que representan una acción dentro del procedimiento. Utilizan unos símbolos normalizados, con los pasos del algoritmo escritos en el símbolo adecuado y los símbolos unidos con flechas, denominadas líneas de flujo, que indican el orden en que los pasos deben ser ejecutados.

Para su elaboración se siguen ciertas reglas:

- 🖱 Se escribe de arriba hacia abajo y de izquierda a derecha
- 🖱 Siempre se usan flechas verticales u horizontales, jamás curvas
- 🖱 Evitar cruce de flujos
- 🖱 En cada paso expresar una acción concreta

Secuencia de flujo normal en una solución de problema

- 🖱 Tiene un inicio
- 🖱 Una lectura o entrada de datos
- 🖱 El proceso de datos
- 🖱 Una salida de información
- 🖱 Un final



VENTAJAS DE USAR FLUJOGRAMAS:

- 🖱️ Rápida comprensión de las relaciones
- 🖱️ Análisis efectivo de las diferentes secciones del programa
- 🖱️ Pueden usarse como modelos de trabajo en el diseño de nuevos programas o sistemas
- 🖱️ Comunicación con el usuario
- 🖱️ Documentación adecuada de los programas
- 🖱️ Codificación eficaz de los programas
- 🖱️ Depuración y pruebas ordenadas de programas.

DESVENTAJAS DE LOS FLUJOGRAMAS:

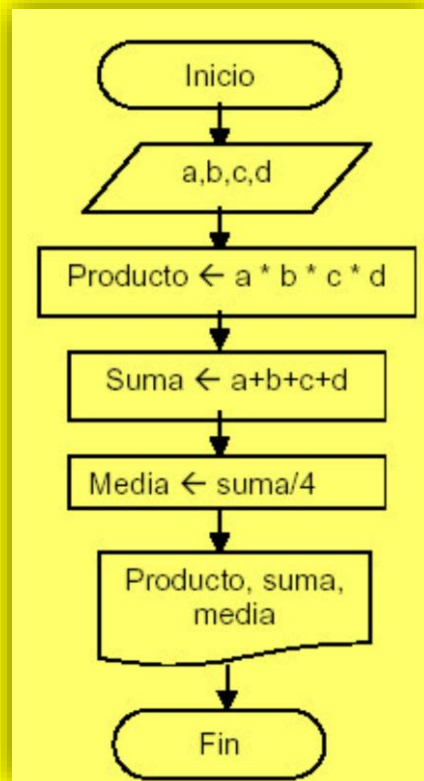
- 🖱️ Diagramas complejos y detallados suelen ser laboriosos en su planteamiento y diseño
- 🖱️ Acciones a seguir tras la salida de un símbolo de decisión, pueden ser difíciles de seguir si existen diferentes caminos
- 🖱️ No existen normas fijas para la elaboración de los diagramas de flujo que permitan incluir todos los detalles que el usuario desee introducir.



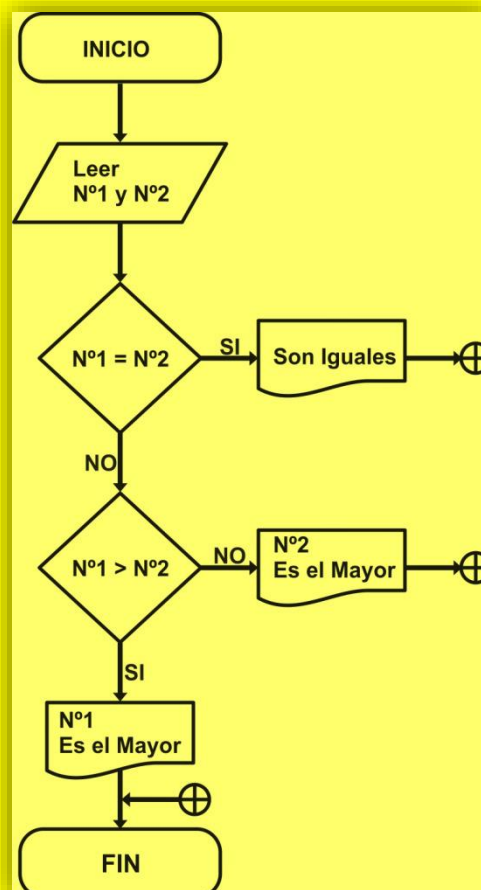
SIMBOLOGÍA PARA DISEÑAR FLUJOGRAMAS:

SÍMBOLO	DESCRIPCIÓN
	Este se utiliza para representar el inicio o el fin de un algoritmo. También puede representar una parada o una interrupción programada que sea necesaria realizar en un programa.
	Este se utiliza para un proceso determinado, es el que se utiliza comúnmente para representar una instrucción, o cualquier tipo de operación que origine un cambio de valor.
	Este símbolo es utilizado para representar una entrada o salida de información, que sea procesada o registrada por medio de un periférico.
	Este es utilizado para la toma de decisiones, ramificaciones, para la indicación de operaciones lógicas o de comparación entre datos.
	Este es utilizado para enlazar dos partes cualesquiera de un diagrama a través de un conector de salida y un conector de entrada. Esta forma un enlace en la misma página del diagrama.
	Este es utilizado para enlazar dos partes de un diagrama pero que no se encuentren en la misma página.
	Este es utilizado para indicar la secuencia del diagrama de flujo, es decir, para indicar el sentido de las operaciones dentro del mismo.
	Este es utilizado para representar la salida de información por medio de la impresora.
	Este es utilizado para representar la salida o para mostrar la información por medio del monitor o la pantalla.

Representando el primer ejemplo como flujograma tenemos:



Ejemplo: Determinar si 2 números son iguales o diferentes, si son diferentes determinar cuál de los 2 es el mayor.



4. CODIFICACIÓN.

Es la conversión de un algoritmo en un programa escribiéndolo en un determinado lenguaje de programación.

PROGRAMACIÓN.

Es un proceso para convertir especificaciones generales de un sistema en instrucciones utilizables por la máquina, que produzcan los resultados deseados. Se le conoce también como desarrollo de software.

PROGRAMA.

Es una lista de instrucciones que la computadora debe seguir para procesar datos y convertirlos en información. Las instrucciones se componen de enunciados usados en lenguajes de programación.

LENGUAJE DE PROGRAMACIÓN.

Es un conjunto de símbolos junto a un conjunto de reglas para combinar dichos símbolos que se usan para expresar programas. Constan de un léxico, una sintaxis y una semántica.

¿Qué conoces tú por léxico, sintaxis y semántica?

LÉXICO: Conjunto de símbolos permitidos o vocabulario

SINTAXIS: Reglas que indican cómo realizar las construcciones del lenguaje

SEMÁNTICA: Reglas que permiten determinar el significado de cualquier construcción del lenguaje.

TIPOS DE LENGUAJES:

Atendiendo al número de instrucciones necesarias para realizar una tarea específica podemos clasificar los lenguajes informáticos en dos grandes bloques:

- ✓ Bajo nivel
- ✓ Alto nivel

LENGUAJE DE BAJO NIVEL.

Es el tipo de lenguaje que cualquier computadora es capaz de entender. Se dice que los programas escritos en forma de ceros y unos están en lenguaje de máquina, porque esa es la versión del programa que la computadora realmente lee y sigue.

LENGUAJES DE ALTO NIVEL.

Son lenguajes de programación que se asemejan a las lenguas humanas usando palabras y frases fáciles de entender.

En un lenguaje de bajo nivel cada instrucción corresponde a una acción ejecutable por el ordenador, mientras que en los lenguajes de alto nivel una instrucción suele corresponder a varias acciones.

LENGUAJES DE PROGRAMACIÓN A UTILIZAR EN EL DESARROLLO DE LA UNIDAD DIDACTICA

VISUAL BASIC 2013

Visual Basic 2013, es un lenguaje de programación integral rápida de desarrollo para crear aplicaciones y permite la integración y el uso cruzado de los lenguajes de programación .Net

¿POR QUÉ VISUAL BASIC 2013?

La respuesta es muy sencilla, el Visual Basic 2013 o superior te permite realizar unos buenos diseños en tus programas, además de que el programa que realices en Visual Basic se podrá correr (ejecutar) en casi todos los sistemas operativos de la Microsoft, y lo mejor, bueno a mi opinión, es mucho más fácil de realizar los códigos, ya que tiene una ayuda de auto complementar.

Con Basic 2013 o superior, usted puede desarrollar distintos tipos de aplicaciones.

- ✓ Aplicaciones de consola para Windows.
- ✓ Aplicaciones gráficas para Windows.
- ✓ Aplicaciones gráficas para web.

APLICACIONES DE CONSOLA.

Las aplicaciones de consola se ejecutan en una ventana de DOS que proporciona una salida basada en texto, por esta razón este tipo de aplicaciones permite iniciar el aprendizaje de un lenguaje de programación, ya que libera al programador de la

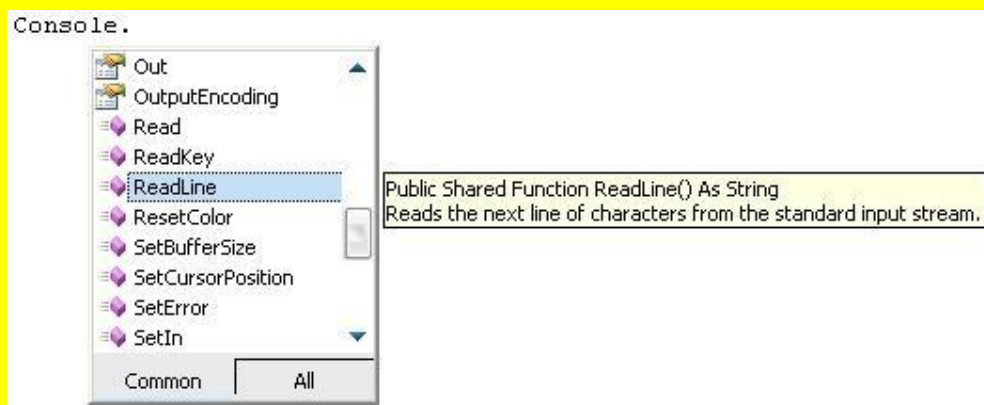
programación de las Interfases Gráficas de Usuarios (IGUs) y centra su atención en los elementos del lenguaje (clases, métodos, condicionales, ciclos, etc.).

¿PORQUÉ CONSOLE APPLICATION Y NO WINDOWS APPLICATION?

Utilizamos la Console Application (Consola de Aplicación) porque ustedes, los que tienen deseos de aprender, les será más fácil de aprender la lógica que van a necesitar para resolver cualquier algoritmo que se les presente, y además porque casi todos los códigos que vamos a usar en la Console Application, lo vamos a usar en Windows Application. Bueno, no uso el Windows Application para comenzar a programar, porque me parece que les enseñaría a manejar moto sin saber manejar bicicleta, en otras palabras, si no aprenden a ver como uno puede capturar y mostrar datos mediante código y no mediante unos controles, al principio les sería más fácil, pero conforme avancemos no les va a quedar en claro, y es por eso que uso primero Console Application, pero no se preocupen, que también nos atreveremos usar Windows Application.

Las palabras reservadas y que funciones que vamos a utilizar para capturar y mostrar datos son:

Console, Esta es la clase que vamos a utilizar, ya que aquí se encuentra las funciones para capturar y mostrar datos. Por ahora vamos a utilizar las siguientes funciones.



Read, nos captura o lee el dato que el usuario ingresa.

Variable = Console.Read

ReadLine, nos captura o lee el dato que el usuario ingresa, pero a la vez hace un cambio de línea al terminar de leer el dato.

Variable = Console.ReadLine

Write, nos permite mostrar ya sea una cadena (texto) o un dato numérico (número).

- ✓ Para mostrar texto y datos calculados o capturados

```
Console.Write("Texto a Mostrar {0},{1}, {2}, Ö, {n}", Var0, Var1, Var2,..., Varn)
```

- ✓ Para mostrar texto

```
Console.Write("Texto a Mostrar")
```

WriteLine, nos permite mostrar ya sea una cadena (texto) o un dato numérico (número), y a la vez hace un cambio de línea, después de haber mostrado el dato.

- ✓ Para mostrar texto y datos calculados o capturados

```
Console.WriteLine("Texto a Mostrar {0},{1}, {2}, Ö, {n}î, Var0, Var1, Var2,..., Varn)
```

- ✓ Para mostrar texto

```
Console.WriteLine("Texto a Mostrar")
```

Como se habrán dado cuenta, que en la función Write y WriteLine, seguido del texto que deseamos mostrar pongo {0}, {1} etc. Pues lo pongo porque para mostrar una variable o un dato seguido del texto que muestro por ejemplo: si deseo mostrar el siguiente texto, seguido de la variable N que contiene un número digamos que 13, deberíamos de escribir el siguiente código.

```
Console.WriteLine("El Valor de la Variable N es : {0}", N)
```

Y lo que nos mostrara en pantalla seria lo siguiente

El Valor de la Variable N es: 13

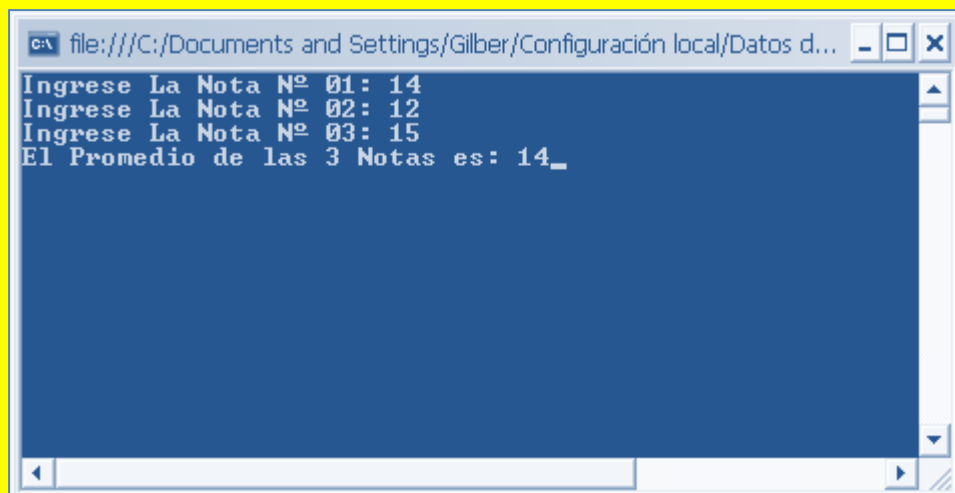
Ejemplo:

1. Se necesita obtener el promedio simple de un estudiante a partir de sus tres notas parciales.

```
Module Ejercicio02

Sub Main()
    'Datos de Salida
    Dim P As Integer
    'Datos de Entrada
    Dim N1, N2, N3 As Integer
    'Ingresar la Nota N° 01
    Console.WriteLine("Ingrese La Nota N° 01: ")
    N1 = Console.ReadLine
    'Ingresar la Nota N° 02
    Console.WriteLine("Ingrese La Nota N° 02: ")
    N2 = Console.ReadLine
    'Ingresar la Nota N° 03
    Console.WriteLine("Ingrese La Nota N° 03: ")
    N3 = Console.ReadLine
    'Calculamos el Promedio de las 3 notas
    P = (N1 + N2 + N3) / 3
    'Mostramos el promedio de las 3 notas
    Console.WriteLine("El Promedio de las 3 Notas es: {0}", P)
    'Esperamos que Presione Cualquier Tecla Para Finalizar el Programa
    Console.ReadLine()
End Sub

End Module
```



```
C:\ file:///C:/Documents and Settings/Gilber/Configuración local/Datos d...
Ingrese La Nota N° 01: 14
Ingrese La Nota N° 02: 12
Ingrese La Nota N° 03: 15
El Promedio de las 3 Notas es: 14_
```

LENGUAJE JAVA

Java es un lenguaje de programación y una plataforma informática comercializada por primera vez en 1995 por Sun Microsystems. Hay muchas aplicaciones y sitios web que no funcionarán a menos que tenga Java instalado y cada día se crean más. Java es rápido, seguro y fiable. Desde portátiles hasta centros de datos, desde consolas para juegos hasta súper computadoras, desde teléfonos móviles hasta Internet, Java está en todas partes.

RAZONES PARA APRENDER JAVA

Qué lenguaje es el más adecuado para aprender a programar, es un tema de discusión entre programadores, profesores de universidad, profesionales, etc. La cuestión es que no hay un criterio unánime respecto a qué lenguaje es el ideal.



De acuerdo con un sondeo de **Hacker News**, Java es uno de los lenguajes de programación más odiados. Sin embargo, no es un argumento de peso odiar Java y que esto signifique que no puedas utilizarlo, es más aún existen muchas razones para usar Java y especialmente como desarrollador.

Es probable que, si no te gusta Java, y estás evitando su aprendizaje, estés ante una lucha equivocada. A continuación, presentamos algunas razones para aprender Java - incluso si lo odias:

1. Las grandes marcas utilizan Java

Los grandes nombres siguen utilizando Java, como, por ejemplo: Google y Amazon usan esta herramienta, por lo que, si quieres tener más competencias utilizar Java en cualquiera de esos lugares, te dará un plus. Ya sabes si quieres un "in", en las empresas, el saber de la herramienta te brindará otra oportunidad. Java es generalizada, y nunca es una pérdida de tiempo para aprender algo que todos los "peces

gordos” están utilizando. Te dará una ventaja - especialmente en un entorno en el que es popular decir que: “odias Java”.

2. Lo necesitas para el desarrollo de Android

Se pueden utilizar algunos lenguajes JVM para desarrollar Android. Sin embargo, el idioma principal para el desarrollo de Android es Java, es más si se quiere desarrollar para Apple el conocer Java te puede ayudar. Existen herramientas de Java para los desarrolladores de Apple también. Pero éstas no siempre están al día. Incluso si usted decide desarrollar para Apple sin Java (iOS 7 promete más funciones para desarrolladores), la realidad es que hay más teléfonos Android, siendo estos mucho más rápido que Apple y superando en términos de aplicaciones descargadas por los consumidores.

3. Conoce más acerca de JVM Idiomas

Puede utilizar más eficazmente lenguajes como Scala y Clojure cuando se tiene una comprensión de Java. Hay mucho que puedes hacer con lenguajes JVM, y la comprensión de Java puede dar un buen lugar para empezar. Además, si usted está aprendiendo las lenguas, que también podría aprender de la fuente.

4. Java aún es avanzado y potente

A pesar de que odies Java, la realidad es que este es un lenguaje de programación con el poder de permanencia, es avanzado y ofrece potentes herramientas para desarrolladores. Puedes utilizar las herramientas integradas de medio ambiente de desarrollo como Eclipse y Netbeans, y también tienes el acceso a la orientación de objetos avanzados de Java, además Java se ha actualizado para que sea más rápido y más cómodo de usar. Por lo que, si no has obtenido recientemente Java, no puedes saber lo que te estás perdiendo.

5. En algún momento tendrás que leer Java

Esto puede ocurrir tarde o temprano incluso si no terminas usando Java, ¿qué pasa si necesitas relacionarte con la herramienta? A pesar de las fallas que presenta Java, aún la herramienta ofrece un tiempo de ejecución más seguro y como también es tan utilizado, hay una gran probabilidad de que, como desarrollador, tengas que leerlo.

VARIABLES

Una variable se define como un identificador que se utiliza para almacenar todos los datos generados durante la ejecución de un programa.

Existen ciertas reglas en cuanto a variables:

- ☞ Claras y con referencia directa al problema.
- ☞ No espacios en blanco, ni símbolos extraños en ellas.
- ☞ Se pueden usar abreviaturas, pero solo de carácter general.
- ☞ No deben ser palabras reservadas del lenguaje.

Ejemplo:

Nombre, Edad, SdoDiario, IngMensual, Perímetro, Calif1, etc.

NOMBRES DE VARIABLES:

El nombre de una variable (o de una constante) tiene que comenzar siempre por una letra y puede tener una longitud hasta 255 caracteres. No se admiten espacios o caracteres en blanco, ni puntos (.), ni otros caracteres especiales.

Los caracteres pueden ser letras, dígitos, el carácter de subrayado (_) y los caracteres de declaración del tipo de la variable (% , & , # , ! , @ , y \$). El nombre de una variable no puede ser una palabra reservada del lenguaje (For, If, Loop, Next, Val, Hide, Caption, And, ...). Para saber cuáles son las palabras reservadas en Visual Basic puede utilizarse el Help de dicho programa, buscando la referencia Reserved Words. De ordinario las palabras reservadas del lenguaje aparecen de color azul en el editor de código, lo que hace más fácil saber si una palabra es reservada o no.

ELECCIÓN DEL TIPO DE UNA VARIABLE:

Si en el código del programa se utiliza una variable que no ha sido declarada, se considera que esta variable es de tipo Variant. Las variables de este tipo se adaptan al tipo de información o dato que se les asigna en cada momento. Por ejemplo, una variable tipo Variant puede contener al principio del programa un string de caracteres, después una variable de doble precisión, y finalmente un número entero. Son pues variables muy flexibles, pero su uso debe restringirse porque ocupan más memoria (almacenan el tipo de dato que contienen, además del propio valor de dicho dato) y requieren más tiempo de CPU que los restantes tipos de variables.

En general es el tipo de dato (los valores que puede tener en la realidad) lo que determina qué tipo de variable se debe utilizar. A continuación, se muestran algunos ejemplos:

- ☞ **Integer** para numerar las filas y columnas de una matriz no muy grande
- ☞ **Long** para numerar los habitantes de una ciudad o los números de teléfonos
- ☞ **Boolean** para una variable con sólo dos posibles valores (sí o no)
- ☞ **Single** para variables físicas con decimales que no exijan precisión
- ☞ **Double** para variables físicas con decimales que exijan precisión
- ☞ **Currency** para cantidades grandes de dinero.

Es muy importante tener en cuenta **que se debe utilizar el tipo de dato más sencillo que represente correctamente el dato real** ya que en otro caso se ocupará más memoria y la ejecución de los programas o funciones será más lenta.

CONSTANTES

Cuando se define un dato de tipo constante, su valor no puede cambiar o verse modificado durante toda la ejecución del programa.

El lugar de la memoria donde está almacenado dicho valor se referencia por el nombre o identificador dado a la constante.

Para declarar un dato como constante únicamente es necesario utilizar la palabra **Const** en la declaración de la variable. Si durante la ejecución se intenta variar su valor se producirá un error.

Ejemplos en Visual Basic:

Const MyVar = 459 Las constantes son privadas por defecto.

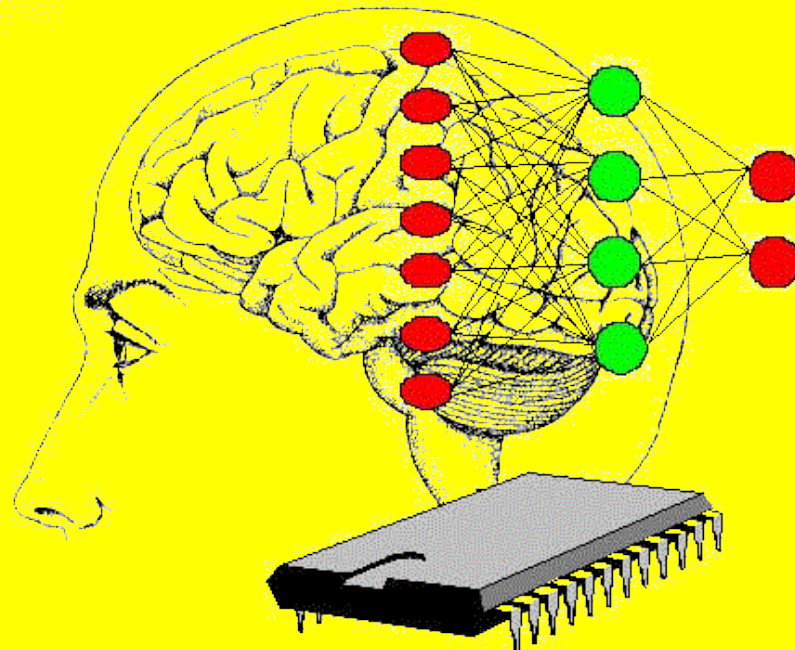
Public Const MyString = "HELP" Declaración de una constante pública.

Private Const MyInt As Integer = 5 Declaración de un entero constante.

Const Str = "Hi", PI As Double = 3.14 Múltiples constantes en una línea.

En Java:

```
class Elipse {  
    final float PI = 3.14159;  
    ...  
}
```



No digas que no tienes suficiente tiempo. Tienes exactamente el mismo número de horas que tuvieron Pasteur, Michelangelo, Helen Keller, Madre Teresa, Leonardo da Vinci, Thomas Jefferson y Albert Einstein

OPERADORES

Los operadores pueden ser un símbolo o palabra que significa que se ha de realizar cierta acción entre uno o dos valores que son llamados operandos.

Tipo	Operación	Operador en Vbasic
Aritmético	Exponenciación	<code>^</code>
	Cambio de signo (operador unario)	<code>-</code>
	Multiplicación, división	<code>*, /</code>
	División entera	<code>\</code>
	Resto de una división entera	Mod
	Suma y resta	<code>+, -</code>
Concatenación	Concatenar o enlazar	<code>& +</code>
Relacional	Igual a	<code>=</code>
	Distinto	<code><></code>
	Menor que / menor o igual que	<code>< <=</code>
	Mayor que / mayor o igual que	<code>> >=</code>
Otros	Comparar dos expresiones de caracteres	Like
	Comparar dos referencias a objetos	Is
Lógico	Negación	Not
	And	And
	Or inclusivo	Or
	Or exclusivo	Xor
	Equivalencia (opuesto a Xor)	Eqv
	Implicación (<i>False</i> si el primer operando es <i>True</i> y el segundo operando es <i>False</i>)	Imp

a. OPERADORES ARITMÉTICOS:

Un operador es un símbolo especial que indica al compilador que debe efectuar una operación matemática o lógica. VISUAL BASIC reconoce los siguientes operadores aritméticos:

Operador	Operación
<code>+</code>	Suma
<code>-</code>	Resta
<code>*</code>	Multiplicación
<code>/</code>	División Flotante
<code>\</code>	División Entera
<code>^</code>	Potencia
MOD	Modulo o Residuo

El Operador (`/`) División Flotante, es el operador de división normal.

El Operador (\) también es división, pero los datos primero, son redondeados al entero más cercano (byte, integer, long) y al final, se trunca la parte residual.

El Operador (^), nos resuelve dos problemas:

☞ Potencias, por ejemplo 3^2 ; se resuelve como:

Dim alfa as integer

alfa = 3^2

Desplegar alfa -----> Sale 9 en pantalla

☞ Raíces solo recordar la ley de exponentes que dice:

Para estos casos:

$$\sqrt[n]{a^m} = a^{m/n}$$

$$\sqrt[3]{5^8} = 5^{8/3}$$

$$\sqrt{9} = 9^{1/2}$$

En Visual Basic el operador (MOD) devuelve el residuo entero de una división entre enteros.

Ejemplo;

Dim alfa As Single;

alfa = 23 MOD 4;

desplegar alfa; <--- El resultado en pantalla es 3

Otro ejemplo;

alfa = 108 MOD 10;

desplegar alfa; <-- El resultado en pantalla es 8

Operadores Aritméticos en JAVA

Operación	sintaxis	significado
Preincremento	++Variable;	Variable = Variable + 1; (antes de asignación)
Postincremento	Variable++;	Variable = Variable + 1; (después de asignación)
Predecremento	--Variable;	Variable = Variable - 1; (antes de asignación)
Postdecremento	Variable--;	Variable = Variable - 1; (después de asignación)

b. OPERADORES DE COMPARACIÓN O DE RELACIÓN:

Los operadores relacionales que reconoce el lenguaje Visual Basic son:

Operador	significado
=	Igual que
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
<>	No es igual que o es diferente que.

En el lenguaje Java:

Operador	Sintaxis	Ejemplo
Menor	<	(EdadJuan < 18)
Menor o igual	<=	(EdadJuan <= EdadPedro)
Mayor	>	(Hipotenusa > 8.0f * 6.2f + 5.7f)
Mayor o igual	>=	(Cateto1 >= Cateto2)
Igual	==	(Contador == 8)
Distinto	!=	(Contador != 8)
Instancia de	instanceof	(Valor instanceof float)

c. OPERADORES LÓGICOS:

En muchas ocasiones es necesario presentar más de una condición para su evaluación al computador.

Por ejemplo, que el computador muestre la boleta de un alumno, si este estudia la carrera de Computación y su promedio de calificaciones es mayor de 70.

Una condición compuesta se define como dos o más condiciones simples unidas por los llamados operadores lógicos.

Los operadores lógicos que Visual Basic reconoce son:

Operador	Significado
And	y
Or	o
Not	no

Los operandos lógicos y sus significados en JAVA son:

Operador	Sintaxis	Ejemplo	Funcionamiento		
Negación	!	Calor = !Frio	!		
			false		true
			true		false
Y	&&	Oportunidad = Bueno && Bonito && Barato	&&	false	true
			false	0	0
			true	0	1
O		Mojado = Llueve Riego		false	true
			false	0	1
			true	1	1

JERARQUÍA DE OPERADORES:

En Visual Basic el problema de no tomar en cuenta la jerarquía de los operadores al plantear y resolver una operación casi siempre conduce a resultados muchas veces equivocados como estos:

Ejemplos:

a. $2 + 3 * 4 = 20$ (incorrecto)

$= 14$ (correcto)

b. si calif1=60 y calif2=80

Entonces si en programa se usa promedio=calif1 + calif2/2 da como resultado promedio = 100

En Visual Basic u otro lenguaje de programación recordar siempre, que antes de plantear una formula en un programa se deberá evaluar contra la siguiente jerarquía de operadores:

- | | |
|-------------------------------|-------------------------|
| 1. () | 7. Suma y resta |
| 2. signo | 8. Concatenación |
| 3. Potencia | 9. Relacionales |
| 4. Producto y división | 10. Negación |
| 5. Div | 11. And |
| 6. Mod | 12. Or |

Datos de tipo **entero** tienen los operadores +, -, *, /, div, mod, abs, sqr, sqrt, ln, exp, sin, cos, tan, pow, etc. Los datos de tipo real tienen los mismos operadores enteros y además trunc, round, int, y otros. La suma y multiplicación de datos de tipo real cumplen la propiedad conmutativa, pero no siempre la asociativa y la distributiva.

Para resolver una expresión aritmética se deben seguir las siguientes reglas:

- ☞ Primero se resuelven las expresiones que se encuentran entre paréntesis.
- ☞ Se procede aplicando la jerarquía de operadores.
- ☞ Al evaluar una expresión, si hay dos operadores con la misma jerarquía, se procede a evaluar de izquierda a derecha.
- ☞ Si hay expresiones relacionales, se resuelven primero paréntesis, luego se encuentran los valores de verdad de las expresiones relacionales y por último se aplica jerarquía de operadores lógicos. En caso de haber iguales, proceder de izquierda a derecha.

Nota: Si se quiere alterar el orden normal de operaciones, entonces usar paréntesis.

Nota: Tampoco es bueno usar paréntesis de más en una operación, esto solo indica que no se evaluó bien la fórmula, como en el siguiente ejemplo;

$$\text{Área} = (\text{base} * \text{altura}) / 2$$

Aquí los paréntesis están de más, porque por orden de operaciones, multiplicación y división tienen la misma jerarquía y entonces se resuelven de izquierda a derecha, en otras palabras, ni que falten paréntesis ni que sobren paréntesis.

EJERCICIOS:

Aplicando la jerarquía de los operadores, encontrar el valor de cada una de las siguientes expresiones:

1. $4 + 1 * 5^2 - 1$
2. $9 / 3 + 4^2 - 5 * 1 + 9 / -2 + 3$
3. $5 / 2 + 3 - 4 * 5 / 2$
4. $(4 + 1) * 5^2 - 1$
5. $17 / 2 + 3^2^2 - 2 * 2 / 2$

Aplicando la jerarquía de operadores, encontrar el valor de verdad de cada una de las siguientes expresiones:

1. Not ((M > N and R > S) or (NOT(T < V and S > M))) Para M=8, N=9, R=5, S=5, T=4 y V=2
2. $(3 * 2^2 - 4 / 2 * 1) > (3 * 2^{-4} / 2 * 1)$ and $(5 > 9 / 3)$

ESTRUCTURAS DE PROGRAMACIÓN

Las **estructuras de programación o control**, denominadas también **sentencias de control**, permiten tomar decisiones y realizar un proceso repetidas veces. Son los denominados bifurcaciones y bucles.

Este tipo de estructuras son comunes en cuanto a concepto en la mayoría de los lenguajes de programación, aunque su sintaxis puede variar de un lenguaje de programación a otro. Se trata de unas estructuras muy importantes ya que son las encargadas de controlar el flujo de un programa según los requerimientos del mismo. Las estructuras de control básicas son:

☞ **If ... Then ... Else**

☞ **Select Case**

☞ **For ... Next**

☞ **Do ... Loop**

☞ **While ... Wend**

☞ **For Each ... Next**

ESTRUCTURA IF ... THEN ... ELSE ...

Esta estructura permite ejecutar condicionalmente una o más sentencias y puede escribirse de **dos formas**.

☞ **La primera** ocupa sólo una línea y tiene la forma siguiente:

If condicion **Then** sentencia1 [**Else** sentencia2]

☞ **La segunda** es más general y se muestra a continuación:

If condicion **Then**

sentencia(s)

[**Else**

sentencia(s)]

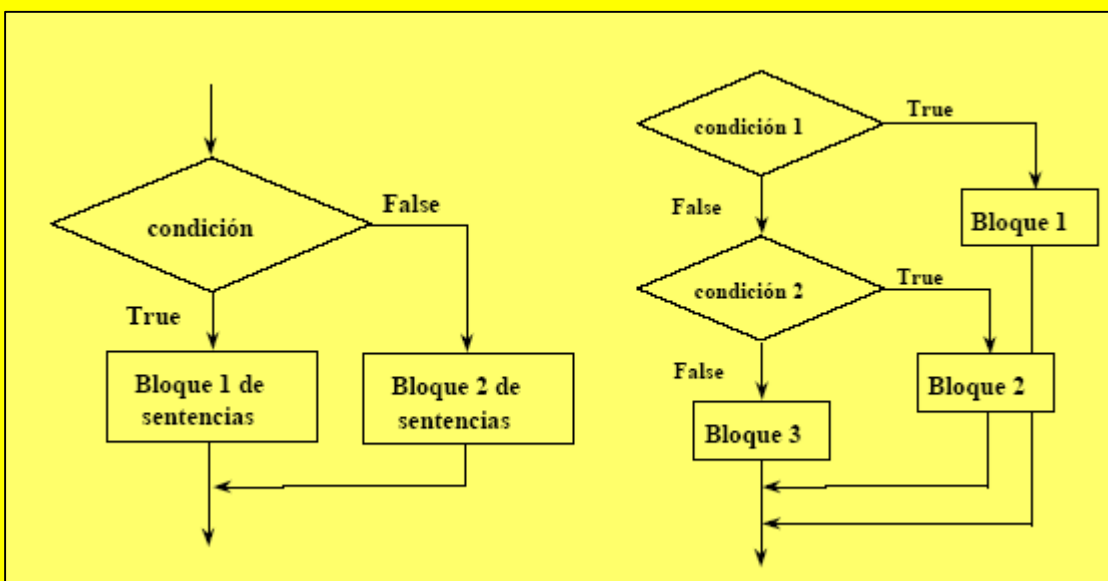
End If

Si **condicion** es **True (verdadera)**, se ejecutan las sentencias que están a continuación de **Then**, y si **condicion** es **False (falsa)**, se ejecutan las sentencias que están a continuación de **Else**, si esta cláusula ha sido especificada (pues es opcional). Para indicar que se quiere ejecutar uno de varios bloques de sentencias dependientes cada uno de ellos de una condición, la estructura adecuada es la siguiente:

```
If condicion1 Then  
sentencias1  
ElseIf condicion2 Then  
sentencias2  
Else  
sentencia-n  
End If
```

Si se cumple la **condicion1** se ejecutan las **sentencias1**, y si no se cumple, se examinan secuencialmente las condiciones siguientes hasta **Else**, ejecutándose las sentencias correspondientes al primer **ElseIf** cuya condición se cumpla. Si todas las condiciones son falsas, se ejecutan las sentencias-n correspondientes a **Else**, que es la opción por defecto.

En la Figura a continuación se presenta esquemáticamente ambas formas de representar estas sentencias:



Ejemplo: Elabora un algoritmo que permita averiguar si una persona debe de inscribirse para el SMO (Servicio Militar Obligatorio), sabiendo su año de nacimiento.

```
Inicio
    Leer AN
    Leer AA
    E = AA - AN
    SI (E > 17) ENTONCES
        Escribir "Debe de Inscribirse en el SMO"
    SINO
        Escribir "No debe de Inscribirse en el SMO"
    FIN_SI
Fin
```

ESTRUCTURA SELECT CASE

Esta sentencia permite ejecutar una de entre varias acciones en función del valor de una expresión.

Es una alternativa a **If ... Then ... ElseIf** cuando se compara la misma expresión con diferentes valores.

Su forma general es la siguiente:

Select Case expresión

Case etiq1

[sentencias1]

Case etiq2

[sentencias2]

Case Else

sentenciasN

End Select

DONDE: *expresion* es una expresión numérica o alfanumérica, y *etiq1*, *etiq2*, ... pueden adoptar las formas siguientes:

1. *expresion*
2. *expresion To expresion*
3. **Is** operador-de-relación *expresion*
4. combinación de las anteriores separadas por coma.

EJEMPLO:

Numero = 8 'Se inicializan las variable.

Select Case Numero 'Se va a evaluar la variable Numero.

Case 1 To 5 'Numero está entre 1 y 5.

Resultado = "Se encuentra entre 1 y 5"

'La siguiente se ejecuta si es True la expresión.

Case 6, 7, 8 ' Numero es uno de los tres valores.

Resultado = "Se encuentra entre 6 y 8"

Case Is = 9 , Is = 10 ' Numero es 9 ó 10.

Resultado = "El valor es 9 o 10"

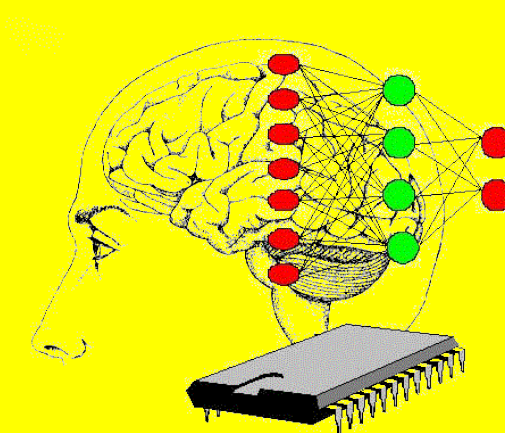
Case Else ' Resto de valores.

Resultado = "El número no se encuentra entre 1 y 10"

End Select

Cuando se utiliza la forma *expresion To expresion*, el valor más pequeño debe aparecer en primer lugar.

Cuando se ejecuta una sentencia **Select Case**, **Visual Basic** evalúa la **expresion** y el control del programa se transfiere a la sentencia cuya etiqueta tenga el mismo valor que la expresión evaluada, ejecutando a continuación el correspondiente bloque de sentencias. Si no existe un valor igual a la **expresion** entonces se ejecutan las sentencias a continuación de **Case Else**.



Nuestra mayor debilidad reside en rendirnos. La forma más segura de tener éxito es intentarlo una vez más

ESTRUCTURA FOR... NEXT

La sentencia **For** da lugar a un lazo o bucle, y permite ejecutar un conjunto de sentencias cierto número de veces. Su forma general es:

```
For variable = expresion1 To expresion2 [Step expresion3]  
[sentencias]  
Exit For  
[sentencias]  
Next [variable]
```

Cuando se ejecuta una sentencia **For**, primero se asigna el valor de la **expresion1** a la variable y se comprueba si su valor es mayor o menor que la **expresion2**. En caso de ser menor se ejecutan las sentencias, y en caso de ser mayor el control del programa salta a las líneas a continuación de **Next**.

Todo esto sucede en caso de ser la **expresion3** positiva. En caso contrario se ejecutarán las sentencias cuando la variable sea mayor que **expresion2**. Una vez ejecutadas las sentencias, la variable se incrementa en el valor de la **expresion3**, o en 1 si **Step** no se especifica, volviéndose a efectuar la comparación entre la variable y la **expresion2**, y así sucesivamente.

La sentencia **Exit For** es opcional y permite salir de un bucle **For ... Next** antes de que éste finalice.

Ejemplo:

```
MyString ="Informática"  
For Words = 3 To 1 Step -1           ' 3 veces decrementando de 1 en 1.  
For Chars = Words To Words + 4      '5 veces.  
MyString = MyString & Chars        ' Se añade el número Chars al string.  
Next Chars                          ' Se incrementa el contador  
MyString = MyString & " "           ' Se añade un espacio.  
Next Words  
  
'El valor de MyString es: Informática 34567 23456 12345
```

ESTRUCTURA DO... LOOP

Un **Loop (bucle)** repite la ejecución de un conjunto de sentencias mientras una condición dada sea cierta, o hasta que una condición dada sea cierta. La condición puede ser verificada antes o después de ejecutarse el conjunto de sentencias. Sus posibles formas son las siguientes:

Formato 1:

```
Do [{While/Until} condicion]
[ sentencias]
[Exit Do]
[ sentencias]
Loop
```

Formato 2:

```
Do
[ sentencias]
[Exit Do]
[ sentencias]
Loop [{While/Until} condicion]
```

La sentencia opcional **Exit Do** permite salir de un bucle **Do ... Loop** antes de que finalice éste.

Ejemplo:

```
Check = True      ' Se inicializan las variables.
Counts = 0
Do      ' Empieza sin comprobar ninguna condición.
Do While Counts < 20    ' Bucle que acaba si Counts >= 20 o con Exit Do.
Counts = Counts + 1    ' Se incrementa Counts.
If Counts = 10 Then    ' Si Counts es 10.
Check = False      ' Se asigna a Check el valor False.
Exit Do      ' Se acaba el segundo Do.
End If
Loop
Loop Until Check = False    ' Salir del "loop" si Check es False.
```

En el ejemplo mostrado, se sale de los bucles siempre con **Counts = 10**. Es necesario fijarse que si se inicializa **Counts** con un número mayor o igual a 10 se entraría en un bucle infinito (el primer bucle acabaría con **Counts = 20** pero el segundo no finalizaría nunca, bloqueándose el programa y a veces el ordenador).

ESTRUCTURA WHILE... WEND

Esta sentencia es otra forma de generar bucles que se recorren mientras se cumpla la condición inicial. Su estructura es la siguiente:

```
While condicion  
[ sentencias]  
Wend
```

Ejemplo:

```
Counts = 0 ' Se inicializa la variable.  
While Counts < 20 ' Se comprueba el valor de Counts.  
Counts = Counts + 1 ' Se incrementa el valor de Counts.  
Wend ' Se acaba el bucle cuando Counts > 19.
```

En cualquier caso, se recuerda que la mejor forma de mirar y aprender el funcionamiento de todas estas sentencias es mediante el uso del **Help** de **Visual Basic**. Ofrece una explicación de cada comando con ejemplos de utilización.

ESTRUCTURA FOR EACH... NEXT

Esta construcción es similar al bucle **For**, con la diferencia de que la variable que controla la repetición del bucle no toma valores entre un mínimo y un máximo, sino a partir de los elementos de un array (o de una colección de objetos). La forma general es la siguiente:

```
For Each variable In grupo  
[ sentencias]  
Next variable
```

FUNCIONES

FUNCIONES DE CONVERSIÓN

Como se habrán dado cuenta que al activar la opción de Option Strict, este nos obliga que tenemos que utilizar funciones de conversión para cada tipo de dato, , las funciones más utilizadas de conversión son las siguientes:

Nombre de la función	Tipo de datos que devuelve	Valores del argumento "expresión"
CBool(expresion)	Boolean	Cualquier valor de cadena o expresión numérica.
CByte(expresion)	Byte	De 0 a 255; las fracciones se redondean.
CChar(expresion)	Char	Cualquier expresión de cadena; los valores deben ser de 0 a 65535.
CDate(expresion)	Date	Cualquier representación válida de una fecha o una hora.
CDbl(expresion)	Double	Cualquier valor Double, ver la tabla anterior para los valores posibles.
CDec(expresion)	Decimal	Cualquier valor Decimal, ver la tabla anterior para los valores posibles.
CInt(expresion)	Integer	Cualquier valor Integer, ver la tabla anterior para los valores posibles, las fracciones se redondean.
CLng(expresion)	Long	Cualquier valor Long, ver la tabla anterior para los valores posibles, las fracciones se redondean.
CObj(expresion)	Object	Cualquier expresión válida.
CShort(expresion)	Short	Cualquier valor Short, ver la tabla anterior para los valores posibles, las fracciones se redondean.
CSng(expresion)	Single	Cualquier valor Single, ver la tabla anterior para los valores posibles.
CStr(expresion)	String	Depende del tipo de datos de la expresión.
	Nota: Todos los objetos de vb.NET tienen unos métodos para realizar conversiones a otros tipos, al menos de número a cadena, ya que tienen la propiedad .ToString que devuelve una representación en formato cadena del número en cuestión (igual que CStr).	
CType(expresion, Tipo)	El indicado en el segundo parámetro	Cualquier tipo de datos
Val(expresion)	Double	Una cadena de caracteres.
Fix(expresion)	Depende del tipo de datos de la expresión	Cualquier tipo de datos
Int(expresion)	Depende del tipo de datos de la expresión	Cualquier tipo de datos

FUNCIONES TIPO FECHA**+ Función Now()**

Esta función nos devuelve la fecha y hora actual del sistema, en un formato largo.

+ Función Date()

Esta función nos devuelve la fecha actual del sistema.

+ Función Year(Fecha)

Esta función nos devuelve el año de una fecha especificada.

+ Función Month(Fecha)

Obtiene el Número de mes de una fecha especificada.

+ Función Day(Fecha)

Obtiene el número de día de una fecha enviada o especificada.

+ Función WeekDay(Fecha)

Obtiene el número del día de la semana, tomando el domingo como valor número 1.

+ Función DateAdd(TipoIntervalo,ValorIntervalo,Fecha)

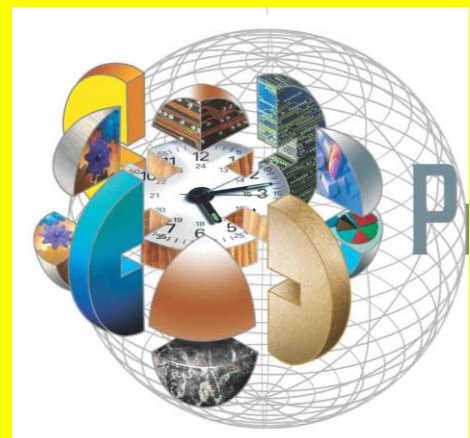
Suma o resta a una fecha, un intervalo determinado, para determinar si se realiza una suma o resta, el TipoIntervalo puede ser días, semanas, horas, meses, años, etc. ValorIntervalo deberá de ser positivo o negativo respectivamente.

+ Función DateDiff(TipoIntervalo,FechaPrimera,FechaSegunda)

Calcula la diferencia existente entre dos fechas.

+ Función DatePart(TipoIntervalo,fecha)

Extrae la parte de una fecha indicada.



CLASE MATH

Permite utilizar funciones matemáticas muy útiles, a continuación veremos un cuadro extraído de la ayuda.

Campos Públicos

E	Representa la base logarítmica natural, especificada por la constante e .
PI	Representa la relación entre la longitud de la circunferencia de un círculo y su diámetro, especificada por la constante π .

Métodos Públicos

Abs	Sobrecargado. Devuelve el valor absoluto de un número.
Acos	Devuelve el ángulo cuyo coseno es el número especificado.
Asin	Devuelve el ángulo cuyo seno es el número especificado.
Atan	Devuelve el ángulo cuya tangente corresponde al número especificados.
Atan2	Devuelve el ángulo cuya tangente es el cociente de dos número especificados.
BigMul	Calcula el producto completo de dos números de 32 bits.
Ceiling	Devuelve el número entero más pequeño mayor o igual que el número especificado.
Cos	Devuelve el coseno del ángulo especificado.
Cosh	Devuelve el coseno hiperbólico del ángulo especificado.
DivRem	Sobrecargado. Devuelve el cociente de dos números y pasa también como parámetro de salida el resto de la división.
Exp	Devuelve e elevado a la potencia especificada.
Floor	Devuelve el número entero más grande menor o igual que el número especificado.
IEEERemainder	Devuelve el resto de la división de dos números especificados.
Log	Sobrecargado, devuelve el logaritmo de un número especificado.
Log10	Devuelve el logaritmo en base 10 de un número especificado.
Max	Sobrecargado. Devuelve el mayor de dos números.
Min	Sobrecargado. Devuelve el menor de dos números.
Pow	Devuelve un número especificado elevado a la potencia especificada.
Round	Sobrecargado. Devuelve el número más próximo al valor especificado.
Sign	Sobrecargado. Devuelve un valor que indica el signo de un número.
Sin	Devuelve el seno del ángulo especificado.
Sinh	Devuelve el seno hiperbólico del ángulo especificado.
Sqrt	Devuelve la raíz cuadrada de un número especificado.
Tan	Devuelve la tangente del ángulo especificado.
Tanh	Devuelve la tangente hiperbólica del ángulo especificado.



PROCEDIMIENTOS

Los procedimientos son las sentencias de código ejecutable de un programa.

Las instrucciones de un procedimiento están delimitadas por una instrucción de declaración y una instrucción End.

Existen tres tipos de procedimientos en Microsoft Visual Basic® .NET: procedimientos Sub, procedimientos Function y procedimientos Property.

- ✚ **Los procedimientos Sub** realizan acciones pero no devuelven un valor al procedimiento que origina la llamada. Los controladores de eventos son procedimientos Sub que se ejecutan en respuesta a un evento.
- ✚ **Los procedimientos Function** pueden devolver un valor al procedimiento que origina la llamada. La instrucción MessageBox.Show es un ejemplo de función.
- ✚ **Los procedimientos Property** devuelven y asignan valores de propiedades de clases, estructuras o módulos.

Un procedimiento puede ser invocado, o llamado, desde otro procedimiento.

Cuando un procedimiento llama a otro procedimiento, se transfiere el control al segundo procedimiento. Cuando finaliza la ejecución del código del segundo procedimiento, éste devuelve el control al procedimiento que lo invocó.

Debido a esta funcionalidad, los procedimientos resultan útiles para realizar tareas repetidas o compartidas. En lugar de escribir el mismo código más de una vez, podemos escribir un procedimiento e invocarlo desde varios puntos de nuestra aplicación o desde otras aplicaciones. Utilizamos un modificador de acceso para definir la accesibilidad de los procedimientos que escribimos (es decir, el permiso para que otro código invoque al procedimiento). Si no especificamos un modificador de acceso, los procedimientos son declarados public de forma predeterminada. Las opciones de accesibilidad para declarar un procedimiento dentro de un módulo:

Public: Ninguna restricción de acceso

Friend: Accesible desde el programa que contiene la declaración y desde cualquier otro lugar del mismo ensamblado

Private: Accesible únicamente en el módulo que contiene la declaración.

Sintaxis

[accesibilidad] Function NombreFuncion[(ListaDeArgumentos)] As TipoDato

Instrucciones

Retorna el Valor de Una Función

End Function

PARÁMETROS O ARGUMENTOS DE LOS PROCEDIMIENTOS:

Cuando queramos que un procedimiento realice una tarea, es posible que necesitemos indicarle alguna información adicional. Esa información se suele indicar mediante parámetros o argumentos. Los argumentos pasados a los procedimientos pueden ser pasados por valor o por referencia.

- ✚ **Por Valor:** Cuando un argumento es pasado por valor, este valor no se podrá alterar, por ejemplo:

```
Module Ejemplo
Sub Main()
MostrarNumero(1)
Console.ReadLine()
End Sub
Sub MostrarNumero(ByVal Valor As Integer)
Valor = Valor + 1
Console.WriteLine("El Valor del Parametro es: {0}", Valor)
End Sub
End Module
```

Explicamos, nosotros llamamos al procedimiento MostrarNumero, y le mandamos el parámetro de 1, este numero 1 entra al procedimiento mediante la variable Valor, que después lo alteramos sumándole mas uno, entonces el valor de la variable Valor seria 2, supuestamente, pero a la hora de mostrarlo en la consola, nos mostrará el valor de 1, se preguntarán ¿Por qué?, esto solo tiene un respuesta, como es pasado por Valor, este Valor no puede ser alterado, por mas que le sumemos, dividamos, restamos, etc. El valor que va a contener la variable Valor siempre va a ser 1.

- ✚ **Por Referencia:** Cuando un argumento es pasado por referencia, este valor se podrá alterar, por ejemplo:

```
Module Ejemplo
Sub Main()
MostrarNumero(1)
Console.ReadLine()
End Sub
Sub MostrarNumero(ByRef Valor As Integer)
Valor = Valor + 1
Console.WriteLine("El Valor del Parametro es: {0}", Valor)
End Sub
End Module
```

Explicuemos, nosotros llamamos al procedimiento `MostrarNumero`, y le mandamos el parámetro de 1, este numero 1 entra al procedimiento mediante la variable `Valor`, que después lo alteramos sumándole mas uno, entonces el valor de la variable `Valor` seria 2, a la hora de mostrarlo en la consola, nos mostrará el valor de 2, se preguntarán ¿Por qué?, esto solo tiene un respuesta, como es pasado por Referencia, este `Valor` puede ser alterado.

ARREGLOS

ARRAY UNIDIMENSIONAL

Un Array (arreglo) es un indicador que puede almacenar varios valores simultáneamente. Cada uno de estos valores se identifica mediante un número al cual se llama índice. Así, para acceder al primer elemento del array habría que usar el índice cero, para el segundo el índice uno, para el tercero el índice dos, y así sucesivamente. Vamos a ver cómo se declara un Array

`Dim Variable(NúmeroDeElementos) As Tipo de Variable`

En efecto, tal como pueden apreciar, el array será utilizable únicamente a partir de su declaración. Por ejemplo:

`Dim Numeros(3) As Integer`

En este ejemplo, el número 3 que está dentro de los paréntesis indica el número total de elementos + 1 de que constará el array. Puesto que todos los arrays están basados en cero, esto es, el primer elemento del array es cero. Por lo tanto, en este caso, el último elemento sería 3, ya que son cuatro y no tres los elementos que lo componen (0, 1, 2 y 3). Veamos un ejemplo algo más completo y después lo comentamos:

`Dim Numeros(3) As Integer`

`Dim K As Integer`

`Numeros(0) = 1`

`Numeros(1) = 2`

`Numeros(2) = 3`

`Numeros(3) = 4`

`For K = 0 to 3`

`Console.WriteLine("Indice [{0}{1}{2}]", K, "=" & Numeros(K))`

`Next`

Ahora veamos la salida en la consola.

Indice [0] = 1

Indice [1] = 2

Indice [2] = 3

Indice [3] = 4

Bueno ahora se ve mas claro lo que hablo, como se habrán dado cuenta al declarar un array e indicar los elementos que va a contener dicho array, siempre debemos de tener en cuenta de esto: Numero de elementos . 1, por ejemplo, si queremos un array de 4 elementos tendríamos que indicarle así: Array (3), por que seria el numero de elementos . 1 en este caso $4 - 1 = 3$.

Aunque es un poco difícil de acostumbrarse, ya que en las versiones antiguas no eran así, se parecen un poco al C, C++.

ARRAY MULTIDIMENSIONAL

Los arrays multidimensionales son aquellos que constan de dos o más dimensiones, es decir, que cada elemento del array viene definido por dos o más índices. Veamos un ejemplo de declaración de un array multidimensional (en este caso, será tridimensional, es decir, con tres dimensiones):

Dim Variable(x,x,x) **As** Tipo de Variable

Como vemos, hay tres letras dentro de los paréntesis, separadas por comas, lo cual indica que el array es tridimensional, puesto que los tres índices del mismo se separan uno de otro por comas. Veamos un pequeño ejemplo que lo clarifique un poco más:

Dim Alumnos(1,3) **As** Integer

Este array es bidimensional y serviría para almacenar una lista de alumnos por Nombres y Apellidos, tenemos un array bidimensional (el primer índice del array nos almacenará los nombres y el segundo índice nos almacenará los apellidos).

Veamos un poco de código y una tabla para que se hagan una idea de cómo se almacena esto:

```
Alumnos(0,0)="José";
Alumnos(0,1)="Luís";
Alumnos(0,2)="Juan";
Alumnos(0,3)="Pedro";
Alumnos(1,0)="Vargas";
Alumnos(1,1)="Moreno";
Alumnos(1,2)="López";
Alumnos(1,3)="Peralta";
```

Esto sería como almacenar los datos en esta tabla:

	Nombres (0)	Apellidos (1)
Alumnos 0	José	Vargas
Alumnos 1	Luís	Moreno
Alumnos 2	Juan	López
Alumnos 3	Pedro	Peralta

Bueno parece que está un poco claro, lo de arrays, en esta tabla he separado por indicie, mejor dicho lo que declaro en la variable lo he graficado, como?, bueno así: tenemos nuestro array llamado Alumnos(1,3), todos sabemos que va a tener dos columnas y cuatro filas, así que en la columna 0 voy a

almacenar los nombres y en la columna 1 sus apellidos, guiándome de los índices de cada nombre que ingrese, en otras palabras en el array Alumnos, en la columna 0 y en la fila 0 voy a almacenar el nombre de José, así que en la columna 1 y en la misma fila 0 voy a almacenar su apellido Vargas, y así sucesivamente hasta que se llene el array.

Ahora veamos unos ejercicios para que lo entiendan mejor.

EJERCICIOS

 Se desea mostrar los cuatros primeros números positivos y el número del índice en donde es almacenado en el array unidimensional.

Solución:

```
Module Ejercicio1
'Ejemplo de un Array Unidimensional
Sub Main()
Dim ArrayUnidimensional(3) As Integer
Dim X, Num As Integer
For X = 0 To 3
    Num = X + 1
    ArrayUnidimensional(X) = Num
Next
Console.WriteLine()
Console.WriteLine("Array Unidimensional 4")
Console.WriteLine()
For X = 0 To 3
    Console.WriteLine("Indice [{0}{1}{2}", X, "] = ", ArrayUnidimensional(X))
Next
Console.ReadLine()
End Sub
End Module
```

Ahora expliquemos, primero declaramos un array unidimensional llamado: ArrayUnidimensional, este va a contener 4 elementos, por lo tanto entre los paréntesis pondremos el numero de elementos menos uno, ósea 3, después declaramos una variable que va a comportarse (funcionar) como un contador, así que primero almacenamos los valores en el array, en el índice 0 tendrá almacenado el valor de 1, en el índice 1 tendrá almacenado el valor 2 y así sucesivamente hasta almacenar los cuatro elementos. Después pasaremos a mostrar elemento por elemento con el for, tomando desde el índice 0 a 3, y eso es todo lo que hace ese código.

 Se desea mostrar los dieciséis primeros números positivos y el número del índice en donde es almacenado en el array bidimensional.

Solución:

```

Module Ejercicio2
'Ejemplo de un Array Bidimensional
Sub Main()
Dim ArrayBidimensional(3, 3) As Integer
Dim X, Y, Num As Integer
For X = 0 To 3
For Y = 0 To 3
Num = Y + 1
If X = 1 Then
Num += 4
Elseif X = 2 Then
Num += 8
Elseif X = 3 Then
Num += 12
End If
ArrayBidimensional(X, Y) = Num
Next
Next
Console.WriteLine()
Console.WriteLine("Array Bidimensional 4 x 4")
Console.WriteLine()
For X = 0 To 3
For Y = 0 To 3
Console.WriteLine("Indice [{0}]{1}{2}{3}{4}", X, "[", Y, "] = ",
ArrayBidimensional(X, Y))
Next
Next
Console.ReadLine()
End Sub
End Module

```

Bueno expliquemos, primero declaramos el array bidimensional que va a ser ArrayBidimensional(3,3), como ustedes saben, este array va a contener 16 elementos y no 9 como parece, después almacenamos los valores mediante un For, mejor dicho, almacenamos los valores mediante una estructura de repetición, primero almacenaremos los valores de la primera fila, así que el array bidimensional será así: cuando X valga 0 será y Y valga 0, ArrayBidimensional(0,0) y almacenaremos el valor de 1, por lo tanto X va a mantener su valor de 0 y Y tendrá el valor de 1, por lo tanto el array será así: ArrayBidimensional(0,1) va a tener el valor de 1 y así sucesivamente hasta que el valor de X sea 1, esto nos indicaría que tocaría llenar la fila 1 por lo tanto cuando X valga 1 y Y valga 0 el array será así: ArrayBidimensional(1,0) va a contener el valor de 5, bueno si están algo confundido, veamos gráficamente como estaría almacenado el array.

		Y			
		0	1	2	3
X	0	1	2	3	4
	1	5	6	7	8
	2	9	10	11	12
	3	13	14	15	16

Gráficamente eso es lo que haría la codificación, con respecto a al almacenar de elementos en el array, y para mostrar los elementos, sería casi lo mismo, por no decir lo mismo, primero muestra el elemento que está en el índice (0,0), y después el del índice (0,1), y así sucesivamente, según lo que indique el For.

Para que les quede un poco mas claro la idea de los array, resolveremos dos ejercicios más de arrays, uno de unidimensional y el otro bidimensional, cada uno respectivamente con su gráfico para que les sea más claro.

- 🖱 Se desea mostrar los Nombres y Apellidos de 4 alumnos que están almacenados en arrays unidimensionales diferentes.
- 🖱 Se desea mostrar los Nombres y Apellidos de 4 alumnos que están almacenados en un array bidimensional.



Y Josué hijo de Nun fue lleno del espíritu de sabiduría, porque Moisés había puesto sus manos sobre él; y los hijos de Israel le obedecieron, e hicieron como Jehová mandó a Moisés.